

Week 12 easier

Week 12: Traditional ML Classifiers. Training SVMs, Decision Trees, and Random Forests on audio features. Dimensionality reduction (PCA).

Week 12 easier

Can you map out two 75 min lectures for week 12. The class is a senior/graduate level DSP and ML course. The textbooks are Oppenheim/Schafer "Discrete-Time Signal Processing" and Raschka "Machine Learning with PyTorch and Scikit-Learn"

Gemini Pro:

Here is a detailed map for two 75-minute lectures designed for a senior/graduate-level audience. This plan bridges the gap between the feature extraction concepts in Oppenheim/Schafer and the traditional machine learning implementations detailed in Raschka.

Lecture 23: Dimensionality Reduction & Maximum Margin Classifiers

Theme: Taming high-dimensional audio data and finding linearly separable boundaries.

Readings: Raschka, Chapter 5 (Compressing Data via Dimensionality Reduction) & Chapter 3 (Support Vector Machines).

This lecture transitions students from the high-dimensional audio features (like MFCCs and spectral centroids) extracted last week into mathematical techniques for reducing that data and classifying it.

Duration	Topic	Key Concepts & Textbook Tie-ins
10 min	The Curse of Dimensionality in Audio	Recap Week 11: When we extract 13 MFCCs, zero-crossing rates, and spectral roll-off across hundreds of audio frames, our feature space explodes. Discuss how distance metrics degrade in high dimensions.
25 min	Principal Component Analysis (PCA)	Theory: Walk through the covariance matrix of our audio features. Explain eigenvalues and eigenvectors (Raschka Ch. 5).

DSP connection: Relate this to orthogonal transformations they know from Oppenheim (like the DCT used in MFCCs). | | **10 min** | **PCA in Practice** | **Application:** Scree plots for explained variance. How many principal components do we need to accurately represent a snare drum vs. a kick drum? | | **20 min** | **Support Vector Machines (SVMs)** | **Theory:** Define the maximum margin hyperplane. Introduce the objective function and the concept of support vectors (Raschka Ch. 3).

Kernels: Introduce the Radial Basis Function (RBF) kernel for non-linearly separable audio classes. | | **10 min** | **Implementation & Wrap-up** | Brief overview of `sklearn.decomposition.PCA` and `sklearn.svm.SVC`. Show a quick code snippet chaining PCA into an SVM pipeline. |

Lecture 24: Tree-Based Models & Ensemble Learning

Theme: Non-linear decision boundaries and combating over-fitting in complex audio spaces. **Readings:** Raschka, Chapter 3 (Decision Tree Learning & Random Forests).

Audio features often have non-linear, hierarchical relationships (e.g., "If the spectral centroid is high AND the zero-crossing rate is high, it's likely a cymbal crash"). Tree-based models capture this human-like reasoning perfectly.

Duration	Topic	Key Concepts & Textbook Tie-ins
10 min	Intuition: Hierarchical Audio Classification	Start with a white-board exercise: Ask students how <i>they</i> would manually classify a trumpet vs. a piano based on frequency domain features. Frame this as a flowchart.
20 min	Decision Trees	Theory: Splitting criteria. Detail Information Gain, Entropy, and Gini Impurity (Raschka Ch. 3).

Visualizing: Show how a decision tree partitions a 2D feature space (e.g., Spectral Spread vs. Energy) into orthogonal decision boundaries. | | **15 min** | **The Over-fitting Problem** | Discuss how deep trees will perfectly memorize the training audio (capturing noise/background artifacts) but fail on unseen test data. Introduce tree pruning. | | **20 min** | **Random Forests & Bagging** | **Theory:** Bootstrap aggregating (Bagging). Explain how combining many weak, over-fitted learners (trees trained on random subsets of audio features and samples) creates a robust, generalized classifier. | | **10 min** | **Feature Importance** | One of the biggest advantages of Random Forests is interpretability. Show

how sklearn can rank which audio features (e.g., MFCC #2 vs. Spectral Flux) were actually the most useful for splitting the dataset. |

Can you design a 90-minute coding lab using Scikit-Learn where students apply PCA and Random Forests to an environmental sound dataset like ESC-50?

Gemini Pro:

Here is a complete blueprint for a 90-minute coding lab using Python, Scikit-Learn, and Librosa. This lab walks students through the entire pipeline: raw audio to extracted features, reduced via PCA, and classified by a Random Forest.

Before they write the code, this interactive sandbox lets them experiment with the hyperparameter tradeoffs they will be coding:

Lab Guide: Environmental Sound Classification

Prerequisites: Students should have `librosa`, `scikit-learn`, `pandas`, and `numpy` installed. They should have the ESC-50 dataset downloaded and unzipped.

1. **Data Loading and Feature Extraction:** 30 Minutes. To process audio in a traditional ML pipeline, we must convert variable-length time-series data into fixed-size feature vectors. We will extract 13 Mel-Frequency Cepstral Coefficients (MFCCs) for each frame, and then compute the mean across all frames to get a single 13-dimensional vector per audio file.

Instructor note: To keep the lab within 90 minutes, have students filter the dataset to just 5 distinct classes (e.g., dog, rain, crying baby, helicopter, chainsaw) rather than all 50.

```
import pandas as pd
import numpy as np
import librosa
import os

# Load metadata
meta = pd.read_csv('ESC-50-master/meta/esc50.csv')

# Filter for just 5 classes for faster training during Lab
target_classes = ['dog', 'rain', 'crying_baby', 'helicopter', 'chainsaw']
df = meta[meta['category'].isin(target_classes)]

features = []
labels = []

for index, row in df.iterrows():
```

```
file_path = os.path.join('ESC-50-master/audio', row['filename'])
# Load audio (sr=22050 is default for Librosa)
y, sr = librosa.load(file_path, sr=None)
```

```
# Extract MFCCs
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
# Average across the time axis to create a 1D vector
mfccs_scaled = np.mean(mfccs.T, axis=0)
```

```
features.append(mfccs_scaled)
labels.append(row['category'])
```

```
X = np.array(features)
y = np.array(labels)
```

1. **Train/Test Split and Standardization:** 15 Minutes. Because PCA is highly sensitive to the scale of the features (it maximizes variance), standardizing the dataset so each feature has a mean of 0 and a variance of 1 is mandatory.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
# Split the dataset 80/20
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# Fit the scaler on the training data ONLY, then transform both
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

1. **Dimensionality Reduction with PCA:** 15 Minutes. While 13 dimensions is relatively small, visualizing it is impossible. Students will use PCA to reduce the feature space to 2 dimensions to visualize if the audio classes are naturally separable.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Reduce to 2 components for visualization
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
```

```
# Plot the results
plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_train_pca[:, 0], y=X_train_pca[:, 1], hue=y_train,
palette='deep')
plt.title('2D PCA of ESC-50 Audio Features')
plt.xlabel(f'Principal Component 1\n({pca.explained_variance_ratio_[0]*100:.1f}%)')
```

```
plt.ylabel(f'Principal Component 2
({pca.explained_variance_ratio_[1]*100:.1f}%)')
plt.show()
```

1. **Training the Random Forest Classifier:** 20 Minutes. Now students train the Random Forest on the full (but scaled) 13-dimensional dataset. Have them experiment with the `n_estimators` (number of trees) and `max_depth`.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Initialize and train the Random Forest
rf_classifier = RandomForestClassifier(n_estimators=100, max_depth=10,
random_state=42)
rf_classifier.fit(X_train_scaled, y_train)

# Predict on the test set
y_pred = rf_classifier.predict(X_test_scaled)
```

1. **Evaluation and Feature Importance:** 10 Minutes. Finally, analyze the model's performance and peer inside the "black box" to see which MFCC coefficient the Random Forest relied on the most.

```
# Print accuracy and classification metrics
print(classification_report(y_test, y_pred))

# Plot feature importances
importances = rf_classifier.feature_importances_
plt.bar(range(1, 14), importances)
plt.title('Random Forest Feature Importances')
plt.xlabel('MFCC Coefficient (1-13)')
plt.ylabel('Importance Score')
plt.show()
```

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schaffer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well. Please render the math equations in LaTeX.

Chat GPT: (worked for 37 min 30 sec)

First approach failed, retry

Chat GPT: (worked for 65 min 11 sec)

Something went wrong, try again

Chat GPT: (medium)

Yes. I created a complete **Week 12 DSP + Machine Learning instructor package** for the two 75-minute classes, aimed at a combined senior/first-year graduate ECE audience and explicitly connecting the DSP perspective of Oppenheim/Schafer with the ML treatment in Raschka.

The main slide deck has **52 slides**, including both lectures, in-class prediction/discussion questions, homework, the weekly quiz, and a separate instructor solution appendix. The Python material uses a consistent synthetic audio dataset with **Kick, Snare, and Cymbal** classes, so students can follow the same features through PCA, SVMs, decision trees, random forests, and feature importance.

Files

- [Download the complete instructor package ZIP](#)
- [Download the 52-slide PowerPoint](#)
- [Lecture 23 — PCA and SVM Jupyter notebook](#)
- [Lecture 24 — Trees and Random Forests Jupyter notebook](#)
- [Standalone Python demonstrations](#)
- [Synthetic audio-feature dataset](#)
- [Package README](#)

Lecture 23 — Dimensionality Reduction & Maximum-Margin Classifiers

The lecture develops the material rather than treating PCA and SVM as black-box sklearn calls. It covers the high-dimensional audio-feature problem, loss of distance contrast, standardization, the covariance matrix, eigenvalues/eigenvectors, PCA geometry, explained variance and scree plots, and a hand-worked 2-D PCA example.

I also make the DSP connection explicit: **PCA versus the DCT used in MFCC processing**. The distinction is useful pedagogically:

The DCT is a fixed orthogonal transform chosen for useful mathematical and signal-processing properties, while PCA learns an orthogonal transform from the covariance structure of the particular dataset.

The SVM portion then develops the maximum-margin hyperplane, hard versus soft margins, the role of (C), support vectors, and the RBF kernel. The lecture ends by putting the whole workflow into a leakage-safe sklearn pipeline:

raw audio features → standardization → PCA → classifier → validation

The notebook includes scaling, PCA, scree plots, PCA loadings, 2-D PCA visualization, linear versus RBF SVMs, GridSearchCV, and proper Pipeline usage.

Lecture 24 — Tree-Based Models & Ensemble Learning

This starts with the requested human-classification/flowchart activity and then turns that reasoning into a mathematical decision tree.

Students work through Gini impurity, entropy, information gain, and an actual split calculation. The slides then show geometrically why trees produce **axis-aligned partitions** in feature space.

The overfitting discussion uses training/test performance versus tree depth, followed by controlled complexity and **cost-complexity pruning**. Random forests are then developed from bootstrap aggregation rather than introduced simply as an API call. I included the variance-reduction idea behind averaging correlated trees and an explanation of why random feature subsets help decorrelate the ensemble.

The final section covers:

- bootstrap sampling
- random feature subsets
- out-of-bag evaluation
- impurity-based feature importance
- permutation importance
- limitations of treating “feature importance” as a causal explanation

The notebook includes tree visualization, overfitting experiments, pruning, random forests, OOB scoring, and both forms of feature importance.

Homework and quiz

The deck contains a **7-problem weekly homework set** covering both conceptual and computational work, including PCA calculations, explained variance, SVM margin reasoning, Gini/information-gain calculations, overfitting/pruning, bagging/random forests, and interpretation of feature importance.

A **10-minute weekly quiz** is also included, with questions on:

- covariance/PCA directions
- explained variance
- maximum-margin SVMs
- support vectors
- the role of (C)
- Gini impurity
- why deep trees overfit
- what random forests randomize
- OOB observations

- interpretation of feature importance

The **student questions appear first**, while the worked homework and quiz solutions are kept in an **instructor appendix** at the end of the deck.

I also ran the standalone code and checked the executed notebooks; both notebooks contain **no execution errors**. The demonstration currently produces approximately **100% PCA+RBF-SVM, 95.7% decision-tree, and 98.8% random-forest test accuracy**, which gives a useful basis for discussing why a single tree can overfit while an ensemble is more stable.

Week12_DSP_ML_PCA_SVM_Trees_RF.pptx does not open in Microsoft, please repair

ChatGPT:

I repaired the PowerPoint by resaving it through a stricter Office Open XML path and then revalidated and rendered all **52 slides**. The repaired version passes the slide integrity test with no detected overflow.

[Download the repaired PowerPoint](#)

The original file was a structurally valid ZIP/PPTX package and LibreOffice could open it, but Microsoft PowerPoint was evidently rejecting something in the original OOXML serialization. The repaired file has normalized internal XML and media relationships while preserving the slide content and layout.

Week 12, Lecture 23, LaTeX math

Explain in more detail what's going on here and what good answers and/or examples are. Please render any math using Latex style.

Chat GPT:

Absolutely. This lecture is doing something more important than simply teaching two algorithms. It is showing students how to move from a **DSP representation of an audio signal** to a **geometric representation suitable for machine learning**.

The conceptual chain of the lecture is:

audio signal → DSP features → high-dimensional feature vector → PCA
→ lower-dimensional geometry → SVM decision boundary.

That progression is explicit in the deck: feature engineering supplies physically meaningful descriptions of the signal, PCA compresses correlated geometry, and the SVM finds a margin-based decision surface.

Below is how I would explain each part in class, including what I would hope to hear from a strong senior/first-year graduate student.

Slides 2–3: Why do we need dimensionality reduction?

The deck starts with the observation that an apparently modest audio problem can become very high dimensional very quickly. A 2-second recording split into roughly 100 frames, with 13 MFCCs per frame, already produces about 1300 feature values before adding spectral centroid, roll-off, zero-crossing rate, flux, energy, deltas, and other descriptors.

There is an important distinction students should understand here.

A **frame-level feature vector** might be something like

$$\mathbf{x}_n = [c_1[n] \quad c_2[n] \quad \cdots \quad c_{13}[n] \quad \text{centroid}[n] \quad \text{ZCR}[n] \quad \text{RMS}[n]]^T.$$

If there are $D = 16$ features and 100 frames, then a recording contains a 100×16 feature matrix.

If we flatten everything, that could become a 1600-dimensional vector. Alternatively, we could summarize each feature over time using its mean, variance, min, max, etc. Even then, the dimensionality can grow rapidly.

The key teaching point is:

More dimensions do not automatically mean more useful information.

MFCCs, spectral centroid, roll-off, spectral flux, and energy often describe overlapping aspects of the same underlying physical phenomenon. The deck appropriately points out that many audio features can therefore be correlated or redundant.

What does “distance loses contrast” actually mean?

Suppose the features have been standardized and are roughly independent. Consider two feature vectors,

$$\mathbf{x}, \mathbf{y} \in \mathbb{R}^D.$$

Their squared Euclidean distance is

$$\|\mathbf{x} - \mathbf{y}\|^2 = \sum_{k=1}^D (x_k - y_k)^2.$$

As D becomes large, we are summing many random contributions. The absolute distance grows, but the **relative variation in distance** becomes smaller.

So the nearest point and farthest point can begin to look surprisingly similar in relative terms.

A useful way to say this to students is:

In low dimensions, “near” and “far” are usually visually and numerically distinct.
In very high dimensions, almost everything can start looking approximately equally far away.

A good student answer to “Why should we care?” would be:

“Because classifiers or clustering algorithms that depend strongly on Euclidean distance may require much more data before those distances become meaningful.”

An even better answer would add:

“And irrelevant or redundant dimensions can contribute distance without contributing class information.”

That is exactly the reason PCA becomes attractive.

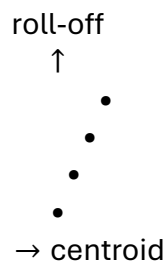
Slides 4–5: What PCA is actually doing

The deck describes PCA as finding orthogonal directions that capture as much variance as possible and projecting the feature vectors onto the first few of those directions.

The most useful way to explain PCA is geometrically first.

Imagine a two-feature dataset with spectral centroid on the horizontal axis and spectral roll-off on the vertical axis.

If those quantities tend to increase together, the point cloud might look like this:



The original coordinate axes are not aligned with the long direction of the cloud.

PCA says:

Rotate the coordinate system so that the first new axis lies along the direction where the data vary most.

That direction is **PC1**.

Then find the next direction, perpendicular to PC1, having the greatest remaining variance. That becomes **PC2**.

This is why I would describe PCA informally as:

Rotate first. Then decide which directions can be discarded.

Students often incorrectly think PCA selects the “best features.” It does not. The deck explicitly warns about this.

For example,

$$PC_1 = 0.42(\text{centroid}) + 0.39(\text{rolloff}) + 0.21(\text{MFCC}_1) + 0.47(\text{MFCC}_2) + \dots$$

would be perfectly normal.

That principal component is a **new feature constructed from the original features**.

So a good answer to

“Which original feature is PC1?”

is:

“Usually none of them. PC1 is a weighted linear combination of the original features.”

Slide 6: Why scaling matters so much

This is a particularly important slide for ECE students because they are used to physical units.

Suppose we have

$$\mathbf{x} = \begin{bmatrix} \text{spectral centroid} \\ \text{RMS amplitude} \\ \text{zero-crossing rate} \end{bmatrix} = \begin{bmatrix} 3500 \text{ Hz} \\ 0.15 \\ 0.12 \end{bmatrix}.$$

If we calculate covariance directly, the feature measured in thousands of Hz can numerically dominate features measured between zero and one.

That does **not** mean spectral centroid is 10,000 times more important.

It simply uses different units.

So we typically standardize feature j as

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

After standardization,

$$\mu_{z_j} \approx 0, \quad \sigma_{z_j} \approx 1.$$

The deck correctly emphasizes that the mean and standard deviation must be estimated from the **training set only**.

A very good class question here is:

“Why can't I standardize the entire dataset before I split it?”

Good answer:

“Because the test data would contribute information to the training transformation. Even though we didn't use the test labels, the test distribution influenced the means and standard deviations.”

Excellent answer:

“That is data leakage. The test set is supposed to simulate completely unseen future observations, so every learned preprocessing operation must be learned without looking at it.”

That same argument will shortly apply to PCA.

Slide 7: Covariance (→) eigenvectors (→) PCA

Now the lecture becomes mathematical.

For N centered observations stored as rows of

$$\mathbf{X} \in \mathbb{R}^{N \times D},$$

the sample covariance matrix is

$$\mathbf{C} = \frac{1}{N-1} \mathbf{X}^T \mathbf{X}$$

as shown in the deck.

The covariance matrix is $D \times D$.

Its diagonal terms are variances:

$$C_{ii} = \text{Var}(x_i),$$

and its off-diagonal terms are covariances:

$$C_{ij} = \text{Cov}(x_i, x_j).$$

Then solve the eigenvalue problem

$$\mathbf{C}\mathbf{v}_k = \lambda_k \mathbf{v}_k$$

with

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D.$$

The deck identifies $\mathbf{v}_k$ as a PCA direction and λ_k as the variance captured in that direction.

The fundamental statement students need to understand is:

$$\text{Var}(\mathbf{X}\mathbf{v}_k) = \lambda_k.$$

So the eigenvalue is not just an arbitrary number coming from linear algebra. It tells us how much the dataset spreads along that principal direction.

The PCA score of an observation is

$$z_{ik} = \mathbf{x}_i^T \mathbf{v}_k.$$

For all observations,

$$\mathbf{z}_k = \mathbf{X}\mathbf{v}_k.$$

A useful connection to Signals and Systems

I would explicitly tell students:

You already know what a change of basis is. PCA is another change of basis.

We are not destroying the data when we rotate into PCA coordinates.

If we retain all D components,

$$\mathbf{X} \leftrightarrow \mathbf{Z}$$

is an invertible orthogonal transformation.

The dimensionality reduction occurs only when we discard some of the coordinates.

Slide 8: The hand calculation

This is one of the best slides to stop and work at the board.

The covariance matrix is

$$\mathbf{C} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Its eigenpairs are

$$\lambda_1 = 3, \quad \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

and

$$\lambda_2 = 1, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

The total variance is

$$\lambda_1 + \lambda_2 = 4.$$

Therefore PC1 explains

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{3}{4} = 75\%$$

of the variance. That is the interpretation presented in the slide.

But I would push it one step further.

Suppose a standardized observation is

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Then

$$z_1 = \mathbf{v}_1^T \mathbf{x} = \frac{1}{\sqrt{2}} (1 + 1) = \sqrt{2},$$

while

$$z_2 = \mathbf{v}_2^T \mathbf{x} = \frac{1}{\sqrt{2}} (1 - 1) = 0.$$

So this observation lies entirely in the “features increase together” direction.

If instead

$$\mathbf{x} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

then

$$z_1 = 0, \quad z_2 = \sqrt{2}.$$

Now the point lies entirely in the **disagreement direction**.

That makes the audio interpretation on the slide much more concrete.

If x_1 is spectral centroid and x_2 is roll-off, then

$$PC_1 \propto \text{centroid} + \text{rolloff}$$

can be interpreted loosely as overall spectral brightness, while

$$PC_2 \propto \text{centroid} - \text{rolloff}$$

describes cases where the two brightness-related measures disagree.

Slide 9: PCA versus the DCT

This is an especially good DSP-to-ML bridge.

The deck makes the distinction that both PCA and the DCT are orthogonal transformations, but the DCT basis is predetermined while the PCA basis is learned from the data.

The DCT can be written conceptually as

$$\mathbf{c} = \mathbf{D}\mathbf{x}$$

where $\mathbf{D}$ is fixed.

For PCA,

$$\mathbf{z} = \mathbf{V}^T \mathbf{x},$$

but

$$\mathbf{V} = [\mathbf{v}_1 \quad \cdots \quad \mathbf{v}_D]$$

is obtained from the covariance matrix of the training data.

An excellent graduate-level observation would be:

“PCA is closely related to the discrete Karhunen-Loève transform because the basis functions are eigenvectors of the covariance matrix.”

That is a useful bridge from classical signal representation to statistical learning.

For MFCCs:

audio $\rightarrow$ STFT $\rightarrow$ power spectrum $\rightarrow$ mel filterbank $\rightarrow$ log $\rightarrow$ DCT $\rightarrow$ MFCCs.

Then we might construct

$$\mathbf{x} = [\text{MFCC}_1, \dots, \text{MFCC}_{13}, \text{ZCR}, \text{centroid}, \text{rolloff}, \text{flux}, \dots].$$

PCA operates on that larger engineered feature vector.

So PCA is **not replacing the DCT in MFCC extraction**. It is operating at another level.

Slide 10: How many PCs should we retain?

Explained variance ratio for component k is

$$r_k = \frac{\lambda_k}{\sum_{j=1}^D \lambda_j}.$$

Cumulative explained variance for the first K components is

$$R_K = \frac{\sum_{k=1}^K \lambda_k}{\sum_{j=1}^D \lambda_j}.$$

You might choose the smallest K such that

$$R_K \geq 0.95.$$

The deck recommends something like 90–95% as a common starting point, followed by downstream validation.

The distinction that students really need is:

$$95\% \text{ variance retained} \neq 95\% \text{ classification accuracy.}$$

A good answer to “Why not?” is:

“Variance describes how well PCA represents the feature distribution. It knows nothing about the class labels.”

Slides 11–12: The most important weakness of PCA

The concept check is excellent because it attacks a common misconception.

Suppose we have two features (x_1, x_2) .

Imagine almost all the variability caused by microphone and performer differences lies along x_1 :

$$x_1 \in [-10, 10].$$

But the actual class information lies along a tiny x_2 displacement:

Class A: $x_2 \approx +0.1$,

Class B: $x_2 \approx -0.1$.

PCA sees enormous variance along x_1 and tiny variance along x_2 .

It may therefore keep x_1 and throw out x_2 .

Result:

We just threw away the feature that perfectly distinguished the classes.

That is precisely the issue identified in the concept-check slide: PCA optimizes variance preservation rather than class separation.

What is a strong student answer?

“PCA is unsupervised. Since it doesn't use y , it cannot know that a low-variance direction is highly discriminative.”

Then ask:

“What would LDA do differently?”

Good answer:

“LDA uses class labels and tries to maximize between-class separation relative to within-class variation.”

That contrast is worth emphasizing:

PCA: preserve variance

versus

LDA: preserve discrimination

Slide 13: Why the sklearn Pipeline matters

The code in the lecture is

```
prep = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
])
```

followed by fitting the transform only using the training set.

Students should understand that PCA itself has learned parameters.

Those parameters include

$$\mu$$

and the eigenvectors

$$\mathbf{v}_1, \mathbf{v}_2, \dots$$

Therefore PCA is **not** simply a fixed mathematical operation like applying a known FFT.

It must be learned.

That means this is wrong:

```
X_scaled = scaler.fit_transform(X)
X_pca = pca.fit_transform(X_scaled)
```

```
X_train, X_test = train_test_split(X_pca)
```

because the test observations helped determine the representation.

Pipeline fixes this during cross-validation because each training fold estimates its own scaler and PCA basis.

That is one of the strongest ML habits students can learn from this lecture.

Slides 14–15: Why SVM instead of “just draw a line”?

Now the lecture changes from representation to classification.

The SVM decision function is

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

and the binary prediction is

$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b).$$

The separating hyperplane is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Many hyperplanes may separate two classes. The SVM asks which one leaves the largest safety zone around the decision boundary. That is the maximum-margin idea introduced in the deck.

For the canonical SVM normalization, the margin boundaries are

$$\mathbf{w}^T \mathbf{x} + b = +1$$

and

$$\mathbf{w}^T \mathbf{x} + b = -1.$$

The distance between them is

$$\frac{2}{\|\mathbf{w}\|}$$

as shown on slide 15.

Therefore maximizing margin is equivalent to minimizing

$$\|\mathbf{w}\|.$$

Conventionally we minimize

$$\frac{1}{2} \|\mathbf{w}\|^2$$

subject to

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1.$$

Why should a larger margin generalize better?

A good student answer would be:

“A large-margin solution is less sensitive to small perturbations of the observations. A small movement of an audio feature vector is less likely to change its class.”

An ECE example is microphone or room variability.

If a snare feature vector moves slightly because the microphone changes, we would prefer it not to cross the classification boundary.

Slide 17: What exactly is a support vector?

The name is meaningful.

For a hard-margin SVM, the critical training samples satisfy

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1.$$

They literally “support” the location of the maximum-margin boundary.

The deck points out that observations far from the margin can move somewhat without altering the solution, while support vectors directly constrain the boundary.

A great in-class thought experiment is:

“I move a kick-drum sample that is very far inside the kick region. What happens?”

Good answer:

“Probably nothing.”

Then:

“I move the kick sample lying right against the margin?”

Good answer:

“The optimal separating hyperplane may move because that observation is a support vector.”

This makes the word *support* intuitive.

Slide 16: Soft margin and C

Real audio will almost never be perfectly separable.

So we introduce slack variables

$$\xi_i \geq 0$$

and solve

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$$

subject to

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$$

which is the formulation shown in the deck.

The best way to explain ξ_i is with three cases.

If

$$\xi_i = 0,$$

the point is correctly classified and on or outside the margin.

If

$$0 < \xi_i < 1,$$

the point is inside the margin but still on the correct side of the decision boundary.

If

$$\xi_i > 1,$$

the point is misclassified.

Then C determines how painful those violations are.

For small C ,

margin violations are relatively inexpensive.

The model is more strongly regularized.

For large C ,

margin violations are expensive.

The classifier tries harder to satisfy the training data.

The deck summarizes this as small C allowing a wider/smooth solution and large C fitting training labels more aggressively.

A useful student shorthand is:

small $C \Rightarrow$ more regularization

large $C \Rightarrow$ less regularization

This sometimes initially feels backwards because students expect a “larger regularization constant” to mean more regularization. In sklearn's SVM convention, it is the opposite.

Slide 18: RBF kernels

The RBF kernel on the slide is

$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$

and this gives a very concrete similarity measure.

If

$$\mathbf{x} = \mathbf{x}',$$

then

$$K = 1.$$

As the vectors become farther apart,

$$K \rightarrow 0.$$

So we can interpret the kernel roughly as:

“How similar is this observation to that observation?”

The parameter γ controls how quickly similarity decays.

For

$$\gamma \ll 1,$$

even points reasonably far apart can remain similar.

The influence is broad and the decision boundary tends to be smooth.

For

$$\gamma \gg 1,$$

similarity dies rapidly with distance.

Each training observation has very local influence, so highly intricate boundaries become possible.

A useful quantitative interpretation comes from asking when

$$K = e^{-1}.$$

Then

$$\gamma r^2 = 1,$$

so

$$r = \frac{1}{\sqrt{\gamma}}$$

gives a rough characteristic radius.

That provides students with a much better intuition than simply saying “gamma controls the kernel.”

How C and γ interact

This is worth adding verbally because students often treat them independently.

Large C says:

“Fit the training samples closely.”

Large γ says:

“Allow very local structure.”

Together,

large C + large γ

can produce an extremely flexible classifier.

That combination is therefore a classic recipe for overfitting.

Conversely,

small C + small γ

can produce excessive smoothing and underfitting.

So there is no universally correct value for either parameter.

They are hyperparameters to be selected using validation.

Slide 19: Why the pretty PCA plot can fool you

The deck appropriately warns students not to infer too much from a two-dimensional visualization.

Suppose PCA gives 12 useful components.

You display only

PC_1, PC_2 .

Classes might overlap badly in that plot while being easily separable in

$(PC_1, PC_2, \dots, PC_{12})$.

The opposite can also happen: the plot might look beautifully separated merely because the example is unusually clean.

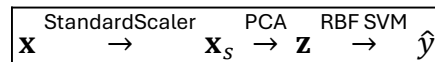
Therefore:

visual separation $\neq$ generalization evidence

The evidence comes from performance on data not used to fit the model.

Slide 20: What the complete pipeline is really saying

The final code is conceptually very clean:



The deck uses

```
model = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
    ("svm", SVC(kernel="rbf", C=3, gamma="scale")),
])
```

and emphasizes tuning C , γ , and possibly PCA dimensionality using cross-validation.

One question I would definitely ask the stronger students is:

“Does PCA necessarily improve SVM classification?”

The correct answer is **no**.

A strong answer is:

“PCA can remove redundancy and noise and reduce computational complexity, but it can also remove discriminative low-variance directions. Whether it helps must be validated.”

That is why the wrap-up correctly writes

scale $\rightarrow$ PCA? $\rightarrow$ classifier

with the conceptual question mark being important.

One additional question worth discussing: what about three audio classes?

The SVM mathematics in the lecture is written for

$$y_i \in \{-1, +1\}.$$

But kick, snare, and cymbal give us three classes.

That is an excellent student question.

A good answer is:

“The fundamental SVM is binary. A multiclass implementation combines multiple binary SVMs.”

In scikit-learn, SVC normally handles the training problem using pairwise, one-versus-one classifiers.

For three classes

$K, S, C,$

the underlying comparisons include

K vs. $S,$

K vs. $C,$

and

S vs. $C.$

So the binary maximum-margin mathematics remains the foundation.

The conceptual story I would want students to leave with

By the end of the 75 minutes, I would not primarily test whether they remember the sklearn function names. I would want them to be able to explain the following argument in their own words:

“DSP gives us meaningful features, but those features may be numerous, differently scaled, and highly correlated. We standardize them so numerical units do not arbitrarily dominate the geometry. PCA rotates the feature space into directions of decreasing variance and can reduce dimensionality by discarding weak directions. However, because PCA does not know the labels, high variance does not necessarily mean high discriminative value. An SVM then finds a decision boundary that maximizes the margin between classes. Soft-margin SVMs allow violations controlled by C , and RBF kernels provide nonlinear boundaries whose locality is controlled by γ . All preprocessing and hyperparameter selection must be performed using training data and cross-validation so the final test set remains genuinely unseen.”

If a senior undergraduate can explain that coherently, they understand the lecture. If a first-year graduate student can additionally derive

$$\mathbf{C}\mathbf{v}_k = \lambda_k \mathbf{v}_k,$$

explain why

$$\lambda_k = \text{Var}(\mathbf{X}\mathbf{v}_k),$$

and connect maximum margin to

$$\min \frac{1}{2} \|\mathbf{w}\|^2,$$

then they are at the mathematical level I would expect for the course.

The deck itself intentionally moves from high-dimensional audio features through PCA and then into maximum-margin and kernel classification, while repeatedly warning students about variance versus discrimination and visualization versus actual validation.

Week 12, Lecture 24, LaTeX math

Please convert all math expressions to LaTeX. Don't change anything else.

Gemini Flash:

Absolutely. Lecture 24 is really about a single progression:

simple rules → decision tree → overfitting → regularization → bagging → random forest.

The main conceptual contrast with Lecture 23 is useful for students:

PCA/SVM: geometric view of feature space

versus

Trees/forests: hierarchical partitioning of feature space.

The deck explicitly frames the lecture around nonlinear decision rules, overfitting, bagging, and feature importance. Below is how I would expand the material in class, including good student answers, examples, and the mathematics behind the slides. Slides 2–3: From human rules to a decision tree The lecture starts by asking students how they would distinguish a trumpet from a piano using engineered audio features. The suggested reasoning uses features such as spectral flux, MFCCs, and spectral shape. This is an excellent starting point because a decision tree is basically a formalized version of a sequence of human questions. For example:

Is spectral flux $> \tau_1$?

If yes, perhaps the signal contains a strong transient attack. Then:

$$\text{Is spectral centroid} > \tau_2?$$

Then:

$$\text{Is MFCC}_3 < \tau_3?$$

Eventually we arrive at a classification. A hypothetical tree might represent:

```
if flux > 0.42
  if centroid > 3200 Hz → cymbal
  else → piano
else → trumpet.
```

The important point is that we do not normally hand-select the thresholds. The training algorithm determines quantities such as

$$\tau_1, \tau_2, \tau_3$$

by finding splits that improve class purity.

A good answer to the opening whiteboard exercise If you ask:

“How might a human distinguish trumpet and piano using our DSP features?” a reasonable student answer would be:

“A piano note usually has a strong attack followed by decay, while a sustained trumpet tone may be more stationary. So spectral flux or frame-to-frame energy change could be useful early in the tree.” A stronger answer:

“After transient behavior, I might examine spectral envelope or MFCCs because harmonic structure and resonances differ between the instruments.” An excellent answer would also say:

“These are plausible physical rules, but we should let the training data determine whether those features and thresholds actually generalize.” That last qualification is important. What a tree is mathematically At every internal node, the tree chooses a feature index j and a threshold τ . The rule has the form

$$x_j < \tau.$$

Then

$$x_j < \tau \Rightarrow \text{left child,}$$

and

$$x_j \geq \tau \Rightarrow \text{right child.}$$

The deck emphasizes that a tree recursively partitions the feature space and that each root-to-leaf path is a logical AND of threshold conditions. For example, one leaf might correspond to

$$(\text{centroid} < 2800) \wedge (\text{flux} > 0.35) \wedge (\text{MFCC}_2 < -1.2).$$

That leaf might predict “snare.” So a decision tree is really creating piecewise regions of the feature space. Slide 4: Gini impurity The deck introduces Gini impurity as a measure of class mixing. For a node containing K classes,

$$G = 1 - \sum_{k=1}^K p_k^2$$

where

$$p_k = \frac{\text{number of samples in class } k}{\text{number of samples in the node}}.$$

Pure node Suppose the node contains

$$[30,0,0].$$

Then

$$p_1 = 1, \quad p_2 = 0, \quad p_3 = 0.$$

Therefore,

$$G = 1 - (1^2 + 0^2 + 0^2) = 0.$$

That is a perfectly pure node. Balanced three-class node Suppose instead we have

$$[10,10,10].$$

Then

$$p_1 = p_2 = p_3 = \frac{1}{3}.$$

Therefore,

$$G = 1 - 3 \left(\frac{1}{3} \right)^2 = 1 - \frac{1}{3} = \frac{2}{3}.$$

That is the value shown in the deck. The conceptual interpretation is:

Low Gini means the node mostly contains one class. High Gini means the classes are mixed. A useful probability interpretation of Gini Students often understand Gini better if you say:

Imagine randomly selecting the class label according to the class proportions in the node. Gini is related to the probability that two independently selected labels disagree. For two classes with probabilities p and $1 - p$,

$$G = 1 - p^2 - (1 - p)^2.$$

Expanding,

$$G = 2p(1 - p).$$

The maximum occurs at

$$p = \frac{1}{2}.$$

Then

$$G_{\max} = 2 \left(\frac{1}{2} \right) \left(\frac{1}{2} \right) = \boxed{\frac{1}{2}}.$$

So a binary node is maximally impure when it is exactly 50/50. Slide 5: Entropy and information gain The second common impurity measure is entropy. For K classes,

$$H = - \sum_{k=1}^K p_k \log_2 p_k$$

with the usual convention

$$0 \log_2 0 = 0.$$

A pure node has

$$H = 0.$$

For two equally likely classes,

$$p_1 = p_2 = \frac{1}{2},$$

so

$$H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}.$$

Since

$$\log_2 \frac{1}{2} = -1,$$

we obtain

$$H = 1 \text{ bit.}$$

For three equally likely classes,

$$H = - \sum_{k=1}^3 \frac{1}{3} \log_2 \frac{1}{3} = \log_2 3 \approx 1.585 \text{ bits.}$$

The deck correctly notes that Gini and entropy often select similar splits; entropy has the cleaner information-theoretic interpretation, while Gini is somewhat simpler computationally. What is information gain? Suppose a parent node is split into left and right children. Let

$$N = N_L + N_R.$$

The weighted child impurity is

$$I_{\text{children}} = \frac{N_L}{N} I_L + \frac{N_R}{N} I_R.$$

Then

$$\boxed{\text{Information Gain} = I_{\text{parent}} - I_{\text{children}}}$$

where I can mean entropy or another impurity measure. For Gini, we often call this an impurity decrease rather than information gain, but mathematically the idea is the same:

$$\boxed{\Delta G = G_{\text{parent}} - \left(\frac{N_L}{N} G_L + \frac{N_R}{N} G_R \right)}$$

The tree searches for the feature and threshold that maximize this reduction. Slide 6: Worked split The deck gives a very nice example: Parent:

20 kick, 20 cymbal.

Candidate split: Left:

18 kick, 2 cymbal.

Right:

2 kick, 18 cymbal.

Let's calculate the entire split.

Parent impurity The parent probabilities are

$$p_{\text{kick}} = p_{\text{cymbal}} = \frac{1}{2}.$$

Thus

$$G_P = 1 - \left[\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right] = \frac{1}{2}.$$

Left child For the left node,

$$p_{\text{kick}} = \frac{18}{20} = 0.9,$$

and

$$p_{\text{cymbal}} = \frac{2}{20} = 0.1.$$

Therefore,

$$G_L = 1 - [(0.9)^2 + (0.1)^2].$$

So

$$G_L = 1 - (0.81 + 0.01) = \boxed{0.18}.$$

The right node is symmetric, so

$$G_R = 0.18.$$

Weighted child impurity Each child contains 20 of the 40 samples. Therefore,

$$G_{\text{children}} = \frac{20}{40}(0.18) + \frac{20}{40}(0.18) = 0.18.$$

So the reduction is

$$\boxed{\Delta G = 0.50 - 0.18 = 0.32}$$

which is quite large. That is why this is a useful split.

Good answer to “Does this threshold help?” “Yes. The parent is maximally mixed, but each child is 90% one class and 10% the other. Gini drops from 0.50 to 0.18, giving an impurity reduction of 0.32.” That is much better than simply saying “yes.” A comparison split worth asking students about Suppose instead we obtain Left:

11 kick, 9 cymbal,

Right:

9 kick, 11 cymbal.

For the left child,

$$G_L = 1 - \left[\left(\frac{11}{20} \right)^2 + \left(\frac{9}{20} \right)^2 \right].$$

Thus,

$$G_L = 1 - (0.3025 + 0.2025) = 0.495.$$

Similarly,

$$G_R = 0.495.$$

Therefore,

$$\Delta G = 0.50 - 0.495 = \boxed{0.005}.$$

So this threshold is essentially useless. That comparison makes tree learning much more intuitive. Slide 7: Why tree boundaries look rectangular The deck notes that each tree split is perpendicular to one feature axis. Suppose the two features are

$$x_1 = \text{spectral centroid}$$

and

$$x_2 = \text{spectral flux}.$$

A split

$$x_1 < 3000$$

creates a vertical line in the (x_1, x_2) plane. Another split

$$x_2 < 0.4$$

creates a horizontal line. Repeated splits create rectangles. The key point is that the overall classifier can be nonlinear, even though each individual boundary is extremely simple. You could approximate a curved boundary using many small rectangles. That is both a strength and a weakness. Strength:

very flexible nonlinear decision regions.

Weakness:

many splits may be required for a smooth diagonal or curved boundary.

This is one reason an RBF SVM and a tree can behave very differently on the same features. An interesting class question Ask:

“Suppose the true decision boundary is exactly $x_2 = x_1$. Which model represents it more naturally: one ordinary decision tree or a linear classifier?” Good answer:

“A linear classifier. A tree would need many axis-aligned steps to approximate a diagonal.” Then ask:

“Does that mean trees cannot model it?” Answer:

“No. They can approximate it increasingly well with more splits, but at the cost of complexity.” Slide 8: Why decision trees do not require scaling This is an important contrast with SVMs. The deck states that standardization is generally unnecessary because a tree depends on sample ordering and thresholds rather than geometric distance. Suppose we have spectral centroid in Hz:

$$x < 3000.$$

Now convert Hz to kHz:

$$x' = \frac{x}{1000}.$$

The corresponding split becomes

$$x' < 3.$$

Every sample goes to exactly the same side. More generally, if

$$x' = f(x)$$

for a strictly increasing transformation f , then

$$x < \tau$$

is equivalent to

$$f(x) < f(\tau).$$

The ordering is unchanged. This is fundamentally different from an RBF SVM, where distance matters:

$$\|\mathbf{x} - \mathbf{x}'\|^2.$$

Changing one feature from units of 0–1 to units of 0– 10^6 dramatically changes Euclidean geometry. Good answer to “Should I StandardScale before a tree?” “Usually it is unnecessary for a conventional decision tree because splits depend only on ordering and thresholds.” Stronger answer:

“Scaling doesn't normally change the possible partitions under monotonic transformations, although other preprocessing issues such as missing values, leakage, or encoding still matter.” That aligns directly with the caution in the deck. Slides 9–10: Why a deep tree overfits The deck's audio-specific framing is very good: a deep tree may start splitting based on microphone hiss, room acoustics, gain, performer identity, or recording-session artifacts rather than the desired source class. This is worth emphasizing because students often think overfitting means:

“The model somehow has too many parameters.” That is true at a high level, but the concrete problem is:

The tree finds accidental patterns that happen to explain the training recordings. Suppose all kick recordings in the training dataset happened to be recorded in Lab A and all snare recordings in Lab B. The model might discover:

60 Hz hum present $\Rightarrow$ kick.

Training accuracy could be excellent. But if a new kick drum is recorded in Lab B, classification fails. The tree learned

recording environment

instead of

instrument characteristics.

Why can a fully grown tree reach almost 100% training accuracy? If we keep splitting until leaves are very small, eventually we can isolate individual observations. A leaf might contain only one sample:

$$N_{\text{leaf}} = 1.$$

Its impurity is necessarily

$$G = 0.$$

Thus a sufficiently flexible tree can often memorize the training set. That does not imply it learned the underlying process. Bias-variance interpretation A shallow tree often has

higher bias

but

lower variance.

A very deep tree tends to have

lower bias

but

higher variance.

So tree regularization is fundamentally a bias-variance tradeoff. A good student explanation is:

“A deep tree is sensitive to small changes in the training set. If a few observations change, the selected splits can change dramatically.” That instability will motivate bagging. Slide 10: Tree regularization controls The deck lists several controls.

`max_depth` Limits how many decisions can occur along a root-to-leaf path. For example,

$$\text{max_depth} = 3$$

means no path contains more than three splitting levels. A smaller depth produces simpler rules. `min_samples_leaf` Requires at least some minimum number of training observations in each terminal leaf. For example,

$$\text{min_samples_leaf} = 10$$

prevents a rule from being created just to explain one or two unusual recordings. This is particularly intuitive for students. Ask:

“Would you trust an audio classification rule supported by one training recording?” Usually no. `min_samples_split` Requires enough observations at a node before the algorithm is even allowed to split it. `max_leaf_nodes` Directly limits the total number of terminal regions. Slide 11: Cost-complexity pruning The deck introduces cost-complexity pruning through α . The useful conceptual objective is

$$R_\alpha(T) = R(T) + \alpha|T|$$

where

$$R(T)$$

represents training error or impurity for tree T , and

$$|T|$$

represents the number of terminal leaves. The two terms compete. The first says:

fit the data.

The second says:

keep the tree simple.

When

$$\alpha = 0,$$

there is no explicit complexity penalty. As α increases, an extra branch must produce a meaningful improvement to justify its existence. A useful analogy to Lecture 23 is regularization. For SVMs, we traded off margin size against classification violations. For pruning, we trade off tree fit against structural complexity. A good pruning example Suppose adding four more leaves decreases training impurity only slightly:

$$R(T_{\text{small}}) = 0.10$$

with

$$|T_{\text{small}}| = 8,$$

and

$$R(T_{\text{large}}) = 0.09$$

with

$$|T_{\text{large}}| = 12.$$

Take

$$\alpha = 0.01.$$

Then

$$R_{\alpha}(T_{\text{small}}) = 0.10 + 0.01(8) = 0.18.$$

For the larger tree,

$$R_{\alpha}(T_{\text{large}}) = 0.09 + 0.01(12) = 0.21.$$

Despite having lower raw training impurity, the larger tree loses after the complexity penalty. That is exactly the intuition behind pruning. Slide 12: Bagging Now we get to the major ensemble idea. The deck describes bootstrap sampling followed by training multiple trees and aggregating their predictions. Suppose the original training set contains

$$N$$

observations. For each tree, we draw

$$N$$

samples with replacement. That phrase is critical. With replacement means the same training example may appear multiple times. For example, if our original indices are

$$\{1, 2, 3, 4, 5\},$$

one bootstrap sample could be

$$\{1, 1, 3, 4, 4\}.$$

Samples 2 and 5 did not appear at all. Why “bootstrap aggregating” becomes “bagging” For each bootstrap dataset,

$$D^{(1)}, D^{(2)}, \dots, D^{(B)},$$

train a model

$$h_1, h_2, \dots, h_B.$$

For regression, average:

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B h_b(\mathbf{x})$$

For classification, use majority vote or averaged class probabilities. For hard voting,

$$\hat{y}(\mathbf{x}) = \text{mode}[h_1(\mathbf{x}), \dots, h_B(\mathbf{x})]$$

The key phrase is:

Bagging does not necessarily make each tree better. It makes the average of many unstable trees more stable. A great classroom voting example Suppose five trees predict:

[kick,kick,snare,kick,snare].

Then the forest prediction is

kick

because kick receives three of the five votes. If probabilities are averaged instead, tree b might provide

$$P_b(\text{kick} \mid \mathbf{x}), \quad P_b(\text{snare} \mid \mathbf{x}), \quad P_b(\text{cymbal} \mid \mathbf{x}),$$

and the forest averages those quantities. Slide 13: Random forest = bagging plus feature randomness The deck gives the essential definition:

Random Forest = Bagging + Random Feature Subsets

Each tree uses a bootstrap sample, and at each split it considers only a random subset of the available features. This second source of randomness is extremely important. Suppose spectral centroid is the strongest individual feature. Without feature randomness, almost every tree may pick spectral centroid near the root. Then all the trees become similar. If the trees are similar, they make similar mistakes. Averaging highly correlated mistakes does not help much. So a random forest sometimes deliberately prevents the strongest feature from being considered. That forces some trees to discover alternatives such as

MFCC₂,

spectral flux,

roll-off,

or

energy decay.

Individually, those trees may be somewhat weaker. Collectively, they may be more useful because their errors are less correlated. This is one of the deepest insights in the lecture. Slide 14: Why averaging reduces variance The deck gives the key result in terms of the number of trees B and the correlation ρ between tree errors. Suppose every tree prediction has variance

$$\sigma^2$$

and pairwise correlation

$$\rho.$$

The variance of the average of B trees is approximately

$$\text{Var}(\bar{h}) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

This equation is worth deriving verbally. There are two pieces. The correlated part is

$$\rho\sigma^2.$$

That does not disappear as B grows. The independent part is

$$\frac{1-\rho}{B}\sigma^2.$$

That shrinks as

$$B \rightarrow \infty.$$

Taking the limit,

$$\lim_{B \rightarrow \infty} \text{Var}(\bar{h}) = \rho\sigma^2$$

So if

$$\rho = 1,$$

all trees make essentially the same errors. Then averaging accomplishes almost nothing. If

$$\rho \approx 0,$$

variance can decrease roughly as

$$\frac{1}{B}.$$

That is why random forests care so much about decorrelating the trees. A good student answer to “Why not just train the same tree 300 times?” “If the data and feature choices are identical, the deterministic tree algorithm will produce essentially the same tree every time. Averaging identical models gives no benefit.” Excellent. Slide 15: Does adding more trees overfit? The deck correctly distinguishes increasing the number of trees from increasing tree depth. This is a subtle but important point. Increasing

$$B$$

does not normally make the forest increasingly flexible in the same way that increasing tree depth does. Instead, a larger B gives a better approximation to the expected ensemble prediction. Therefore:

$$B = 10$$

may be noisy,

$$B = 100$$

more stable, and

$$B = 1000$$

even more stable. Eventually the gain becomes negligible. That is why performance tends to plateau. The practical downside of more trees is mostly computational:

training time,

memory,

prediction time.

Important nuance It would be too strong to say:

“Random forests cannot overfit.” They certainly can. It is better to say:

“Increasing the number of trees usually does not cause the same kind of overfitting produced by making individual trees increasingly complex.” That distinction is exactly what the slide is trying to communicate. Slide 16: Why approximately 36.8% of samples are out-of-bag This is one of the nicest little probability derivations in the lecture. The deck states that a bootstrap sample leaves out roughly 36.8% of the training examples for a particular tree. Here's where the number comes from. Suppose there are

$$N$$

training examples. The probability that one particular sample is not selected on one bootstrap draw is

$$1 - \frac{1}{N}.$$

We make

$$N$$

draws. Therefore, the probability that this observation is never chosen is

$$P(\text{not selected}) = \left(1 - \frac{1}{N}\right)^N.$$

As

$$N \rightarrow \infty,$$

we use the famous limit

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = e^{-1}$$

and

$$e^{-1} \approx 0.3679.$$

Therefore approximately

$$36.8\%$$

of the training observations are out-of-bag for any particular tree. Equivalently, the expected fraction appearing at least once is approximately

$$1 - e^{-1} \approx 0.6321,$$

or

$$63.2\%.$$

This surprises students because the bootstrap dataset still contains N entries. The reason is duplication. How OOB prediction works Suppose training example i was absent from trees

$$\{7, 12, 18, 21, \dots\}.$$

Those trees never trained on observation i . So we can predict observation i using only those trees. Then repeat for every training observation. The result is an internal generalization estimate. A good answer to

“Why isn't the OOB score just training accuracy?” is:

“Because each observation is evaluated only by trees whose bootstrap samples did not include that observation.” Exactly. Slide 17: Feature importance The deck distinguishes two notions:

Mean Decrease in Impurity, or MDI. Permutation importance. These answer related but different questions. Mean decrease in impurity Whenever feature j is used in a split, the split produces an impurity decrease. Suppose a node t produces

$$\Delta I_t.$$

Feature importance can accumulate these decreases across all nodes using feature j , usually weighted by how many samples reach the node, and then average across trees. Conceptually,

$$\text{MDI}_j \propto \sum_{t: \text{split feature} = j} w_t \Delta I_t$$

So if spectral flux is repeatedly used to create very pure child nodes, it will receive high MDI importance. Permutation importance Suppose the trained model has baseline validation accuracy

$$A_{\text{base}}.$$

Now shuffle only feature j . The shuffled feature no longer preserves its relationship with the labels, but every other feature remains intact. Calculate the new accuracy

$$A_j^{\text{perm}}.$$

Then

$$I_j^{\text{perm}} = A_{\text{base}} - A_j^{\text{perm}}$$

If accuracy collapses when MFCC 2 is shuffled, MFCC 2 was important to prediction. If accuracy barely changes, the model did not depend heavily on it—or another correlated feature may be supplying the same information. A very important correlated-feature example Suppose

$$x_1 = \text{spectral centroid}$$

and

$$x_2 = \text{spectral roll-off}$$

are highly correlated. Both encode spectral brightness. The forest may split mostly on x_1 , making x_2 appear unimportant. But that does not imply roll-off contains no information. Alternatively, if you permute x_1 , the model may still use x_2 , so permutation importance of x_1 appears smaller than expected. This is what the deck means when it warns that correlated features can “share or mask” importance. Feature importance is not causality This is worth making very explicit. Suppose microphone model is highly predictive of instrument class because of a biased dataset. The forest may rank microphone-related features very highly. That does not mean

$$\text{microphone} \rightarrow \text{instrument identity}.$$

It means

microphone information helped predict the label in this dataset.

Prediction and causation are completely different questions. So a good student statement is:

“Random-forest importance tells us what the model used successfully for prediction under this data distribution. It does not prove that the feature physically causes the class.”

Slide 18: Tree vs. forest vs. SVM This slide is especially useful because students tend to ask:

“Which classifier is best?” The correct engineering answer is:

It depends on the data and the goal.

The deck compares a readable but high-variance single tree, a robust nonlinear random forest, and a scaled margin-based SVM requiring C and γ tuning. I would frame it this way.

Single decision tree Advantages:

interpretable,
fast,
minimal preprocessing.

Disadvantages:

high variance,
unstable,
axis-aligned boundaries.

Use it when explaining the decision process matters greatly. Random forest Advantages:

powerful nonlinear model,
less sensitive than one tree,
handles mixed feature scales naturally,
usually strong for engineered/tabular features.

Disadvantages:

less transparent,
larger memory footprint,
importance measures require careful interpretation.

RBF SVM Advantages:

strong geometric classifier,
good in moderate-to-high-dimensional spaces,
smooth nonlinear decision surfaces.

Disadvantages:

requires scaling,
 C requires tuning,

γ requires tuning,

and interpretation is more difficult. A good class question “Our features are 13 MFCCs, centroid, flux, roll-off, ZCR, and RMS. We have 800 labeled audio clips. Which classifier should we use?” Weak answer:

“Random forest.” Better answer:

“We don't know yet.” Best answer:

“A tree, random forest, and SVM make different assumptions. We should compare them with the same train/test split and leakage-safe cross-validation, tune each model appropriately, and then inspect both performance and failure modes.” That is exactly the engineering mindset the deck recommends. Slide 19: Understanding the actual code The tree is configured as

```
tree = DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_leaf=8,  
    random_state=1  
)
```

The most important parameters are

$\text{max_depth} = 4$

and

$\text{min_samples_leaf} = 8$.

Those explicitly prevent the tree from growing down to one-recording leaves. The forest is

```
forest = RandomForestClassifier(  
    n_estimators=300,  
    max_features="sqrt",  
    min_samples_leaf=2,  
    oob_score=True,  
    random_state=1,  
    n_jobs=-1  
)
```

as given in the deck. Here the parameters have physical interpretations. $n_estimators=300$

$B = 300$

trees. The goal is stable ensemble averaging. $\text{max_features}="sqrt"$ If there are

D

features, approximately

$\sqrt{D}$

features are considered at each split. If

$$D = 25,$$

then roughly

$$\sqrt{25} = 5$$

features are candidates at a split. This intentionally makes the trees less correlated. min_samples_leaf=2 Avoids leaves supported by only one observation. oob_score=True Computes an internal OOB generalization estimate. n_jobs=-1 Use the available CPU cores to train trees in parallel. This is possible because individual trees are largely independent during training. A very useful DSP example for this lecture Suppose our feature vector is

$$\mathbf{x} = [\text{RMS}, \text{ZCR}, \text{centroid}, \text{rolloff}, \text{flux}, \text{MFCC}_1, \dots, \text{MFCC}_{13}].$$

For kick, snare, and cymbal, a plausible qualitative pattern could be:

Kick drum

low centroid,
lower ZCR,
strong low-frequency energy.

Cymbal

high centroid,
high ZCR,
broadband high-frequency energy.

Snare Often intermediate or mixed, with

strong transient energy

and significant broadband content. A tree might therefore learn something conceptually like

centroid < 1800 Hz → kick,

otherwise test

ZCR > 0.18 → cymbal,

otherwise

snare.

Students should understand that the actual learned values are data-dependent; this is just a physically plausible example. One subtle question worth adding: Are trees “weak learners”? Your slide description talks about combining many learners, but I would be careful with terminology. In boosting, we often deliberately combine weak learners. Random forests often use deep, individually strong but high-variance trees. So I would tell students:

Bagging/random forests are not primarily successful because every tree is weak. They are successful because individual trees are unstable and not perfectly correlated. That distinction is useful at the graduate level. Another subtle point: Bagging mostly attacks variance. This connects beautifully to statistics. If a model has high variance, changing the training set causes large changes in the learned function. Trees are famous for this. Bagging averages over many perturbed training sets, approximated by bootstrap samples. Thus the primary benefit is

variance reduction.

It does not necessarily reduce bias substantially. That is why bagging is particularly effective for unstable learners such as trees. A sequence of good concept-check questions. These would fit naturally throughout the lecture.

Question 1 A node contains

[25,0,0].

What is its Gini impurity? Good answer:

$$G = 0.$$

Why?

The node contains only one class. Question 2 A binary node contains

[10,10].

What is its Gini impurity?

$$G = 1 - \left[\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right] = \boxed{0.5}.$$

Question 3 A split reduces parent Gini from

0.48

to weighted child Gini

0.21.

What is the impurity decrease?

$$\Delta G = 0.48 - 0.21 = 0.27$$

Larger is better. Question 4 Why does a tree not care if spectral centroid is expressed in Hz or kHz? Good answer:

Because a monotonic rescaling preserves sample ordering; the threshold simply rescales with the feature. Question 5 Training accuracy is

99.8%

and validation accuracy is

74%.

What is the likely issue? Good answer:

Strong overfitting/high variance. Possible remedies:

smaller max depth,

larger min samples per leaf,

pruning,

or an ensemble. Question 6 Why is feature randomness useful in a random forest? Good answer:

“It prevents every tree from repeatedly choosing the same dominant features, reducing correlation among the tree predictions.” Question 7 Would

$B = 1000$

identical trees be better than one? Answer:

No. Averaging identical predictions changes nothing. Question 8 Why can OOB data be used for evaluation? Good answer:

“For any specific training sample, some trees did not see it during bootstrap training, so those trees provide an approximately held-out prediction for that sample.” Question 9 If permutation importance for spectral_rolloff is small, does that prove roll-off contains no useful information? Answer:

No. A correlated feature such as spectral centroid may contain essentially the same information. The larger connection to Lecture 23 I would end by explicitly juxtaposing the two lectures.

PCA PCA asks:

Which directions contain the most feature variance?

It creates linear combinations of features. SVM SVM asks:

Which geometric boundary gives a robust margin?

Decision tree A decision tree asks:

Which feature threshold most improves class purity right now?

Random forest A random forest asks:

Can we average many diverse trees to reduce prediction variance?

That makes the Week 12 progression particularly coherent. What I would expect students to be able to say at the end For the senior undergraduate level, a good final explanation would be:

“A decision tree recursively splits one feature at a time to create increasingly pure class regions. Gini impurity or entropy measures how mixed a node is. Deep trees can memorize training artifacts, so we regularize them using depth limits, minimum leaf sizes, or pruning. Bagging trains trees on different bootstrap samples and averages their predictions. A random forest adds random feature selection to decorrelate the trees, which usually reduces variance and improves generalization. Feature importance can tell us what helped prediction but should not be interpreted causally.” For the first-year graduate level, I would also expect students to be comfortable with

$$G = 1 - \sum_k p_k^2,$$

$$H = - \sum_k p_k \log_2 p_k,$$

$$\Delta I = I_p - \frac{N_L}{N} I_L - \frac{N_R}{N} I_R,$$

the pruning objective

$$R_\alpha(T) = R(T) + \alpha|T|,$$

the OOB result

$$\left(1 - \frac{1}{N}\right)^N \rightarrow e^{-1} \approx 0.368,$$

and the ensemble variance relationship

$$\text{Var}(\bar{h}) \approx \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2.$$

Those equations capture most of the mathematical substance behind Lecture 24, while the audio examples connect the mathematics back to the physical features students created earlier in the course. The deck closes with the same synthesis: a tree gives greedy

hierarchical threshold rules, regularization prevents memorization, the forest reduces variance by averaging decorrelated trees, and feature importance is predictive rather than causal.